Tarea Individual DG_S4

Formulación y Búsqueda Ciega en Espacios de Estados

Escuela Politécnica Nacional

Jair Lezcano

Enlace al repositorio de GitHub: jairdavid06/IA-Trabajos

1. Objetivo General

Modelar matemáticamente el problema clásico de inteligencia artificial de los Misioneros y Caníbales, abstraerlo a un script en Python mediante estructuras de control y estados discretos, e implementar estrategias de búsqueda ciega como BFS y DFS que permitan evaluar y comparar empíricamente su rendimiento computacional, optimalidad y eficiencia.

1.1. Objetivos Específicos

- Formalizar teóricamente el problema, detallando con precisión matemática el conjunto de estados, el estado inicial, meta, acciones permitidas y la función de transición sujeto a restricciones.
- Diseñar la estructura lógica en Python, construyendo funciones basadas en tuplas para representar los estados del sistema y filtrar los movimientos seguros en ambas orillas del río.
- Integrar los algoritmos de búsqueda implementando correctamente una cola (FIFO) para la estrategia BFS y una pila (LIFO) para DFS, asegurando la correcta reconstrucción de la ruta de solución.
- Ejecutar una evaluación midiendo y contrastando métricas clave de rendimiento como el tiempo de ejecución a nivel de microsegundos, el número de nodos expandidos, la memoria requerida y el factor de ramificación efectivo.

2. Introducción

En la disciplina de la Inteligencia Artificial, la formalización y resolución de problemas mediante la exploración de espacios de estados constituyen un mecanismo fundamental para comprender cómo los agentes inteligentes planifican acciones bajo restricciones lógicas y uno de muchos ejemplos es el dilema de los Misioneros y Caníbales, permite ver un entorno acotado pero riguroso para modelar matemáticamente dominios complejos, definir estados iniciales, metas, y establecer funciones de transición seguros.

El presente trabajo aborda la abstracción y codificación de este problema clásico utilizando Python, estructurando un entorno que separa por partes el problema junto a

sus restricciones. Específicamente, se implementan los algoritmos de búsqueda en anchura (BFS) y búsqueda en profundidad (DFS), empleando estructuras de datos fundamentales como colas y pilas respectivamente para recorrer el árbol de estados y reconstruir las trayectorias de solución válidas.

Para validar el comportamiento de ambas estrategias, se lleva a cabo un benchmarking que permite contrastar métricas clave como el tiempo de ejecución a nivel de microsegundos, el número de nodos expandidos, el volumen de estados descubiertos en memoria y el factor de ramificación efectivo (b^*). Con base en estos resultados, el informe trata de explicar por qué BFS garantiza el camino óptimo a costa de un mayor consumo de memoria, frente a las limitaciones de optimalidad y el comportamiento de DFS.

3. Formulación de Problema

3.1. Descripción del Problema

El dilema de los Misioneros y Caníbales representa un problema de espacio de estados dentro de la Inteligencia Artificial, encuadrado en la categoría de problemas de planificación y búsqueda ciega. La situación plantea un escenario en el cual un grupo compuesto por tres misioneros y tres caníbales se encuentra en la orilla izquierda de un río y necesita trasladarse hacia la orilla derecha. Para completar este cruce, disponen de una única embarcación con capacidad limitada para alojar a un máximo de dos personas a la vez y un mínimo de una, ya que el bote no puede desplazarse de manera autónoma.

La dificultad central del problema reside en una restricción donde menciona que en ningún momento y en ninguna de las dos orillas del río la cantidad de caníbales puede superar a la de misioneros, a menos que en dicha orilla no exista ningún misionero presente. Si en algún instante los caníbales superan numéricamente a los misioneros en una orilla, los misioneros serán devorados, lo que invalida de inmediato el estado del sistema.

Bajo este contexto, el desafío radica en determinar una secuencia ordenada y finita de movimientos seguros de ida y vuelta que permita transportar exitosamente a las seis personas al otro lado del río. El problema exige equilibrar constantemente el flujo de pasajeros en el bote para evitar trampas lógicas, ciclos infinitos de ida y vuelta o la incursión en estados de peligro, convirtiéndolo en un modelo idóneo para probar y contrastar la eficiencia temporal, espacial y la capacidad de reconstrucción de rutas de los algoritmos de búsqueda ciega.

3.2. Conjunto de Estados

El conjunto de estados, denotado formalmente como S, representa el universo de todas las configuraciones posibles que puede adoptar el sistema durante el proceso de planificación y cruce del río. Para modelar este dominio de manera eficiente y compacta, cada estado se define mediante una tupla tridimensional de la forma (m_{izq}, c_{izq}, b), donde cada elemento captura una variable clave del entorno en un instante determinado:

1. **Número de misioneros en la orilla izquierda (m_{izq}):** Representa la cantidad de misioneros presentes en la orilla de origen. Debido a la población total del problema, este valor puede tomar cualquier número entero en el rango cerrado de 0 a 3.
2. **Número de caníbales en la orilla izquierda (c_{izq}):** Representa la cantidad de caníbales presentes en la orilla de origen. Al igual que los misioneros, su valor pertenece al conjunto de enteros $\{0,1,2,3\}$.
3. **Posición del bote (b):** Una variable binaria que indica la ubicación física de la embarcación. Toma el valor de 1 si el bote se encuentra en la orilla izquierda (origen) y el valor de 0 si se encuentra en la orilla derecha (destino).

Cabe destacar que no es necesario incluir explícitamente a las poblaciones de la orilla derecha dentro de la tupla del estado, ya que estas se deducen de forma determinista mediante el principio de conservación de la población total: los misioneros en la derecha equivalen a $3 - m_{izq}$ y los caníbales en la derecha a $3 - c_{izq}$.

3.2.1. Dimensión teórica del espacio de estados

Desde una perspectiva combinatoria pura, el espacio de estados matemático total se calcula multiplicando las posibles variaciones de sus tres componentes: 4 (misioneros) $\times$ 4 (caníbales) $\times$ 2 (bote), lo que da un total de 32 combinaciones matemáticas posibles. Sin embargo, es fundamental resaltar que no todas estas 32 combinaciones forman parte del espacio de estados válidos del problema.

Gran parte de ese conjunto teórico se descarta de inmediato porque infringe las restricciones o representa configuraciones lógicamente inaccesibles según las reglas de transición del bote. Por lo tanto, los algoritmos de búsqueda operan únicamente sobre un subconjunto reducido y seguro de estados, evitando caminos mortales y optimizando la exploración del árbol de búsqueda.

3.3. Estado Inicial y Estado Meta

En la teoría de búsqueda y planificación por espacios de estados, la definición rigurosa del punto de partida y de la meta deseada permite establecer los límites del problema y determinar cuándo un algoritmo ha completado con éxito su tarea. Para el caso de los Misioneros y Caníbales, estos puntos se definen de la siguiente manera:

- **Estado Inicial (S_0):** Se define formalmente como la tupla $(3,3,1)$. Esta configuración representa el punto de partida absoluto de la planificación, donde los tres misioneros y los tres caníbales se encuentran completamente reunidos en la orilla de origen (izquierda), y la embarcación también se sitúa en dicha orilla, lista para emprender el primer viaje. Como consecuencia del modelo de conservación, la orilla derecha se encuentra inicialmente vacía de personas ($(0,0)$).
- **Estado Meta (S_{IsGoal}):** Se define formalmente como la tupla $(0,0,0)$. Esta configuración representa la meta final y exitosa del proceso de búsqueda. Al alcanzar este estado, se garantiza que tanto los tres misioneros como los tres caníbales han cruzado por completo hacia la orilla derecha, y el bote se encuentra también en la orilla de destino. Cualquier ruta o secuencia de acciones que logre transformar con éxito el estado inicial en este estado objetivo se considera una solución válida al problema.

3.4. Acciones Permitidas

Es el conjunto de acciones u operadores legales, denotado como A, define los movimientos de transporte posibles que los agentes pueden realizar en cada paso. Dado que la embarcación cuenta con una capacidad física limitada y requiere obligatoriamente de al menos un tripulante para navegar de una orilla a otra, los desplazamientos no son arbitrarios.

Las restricciones operativas establecen que el bote puede transportar un máximo de dos personas y un mínimo de una persona por cada viaje ya sea de ida hacia la derecha o de retorno hacia la izquierda. A partir de estas reglas físicas, el conjunto de acciones posibles se modela mediante tuplas de la forma (m_{viaje}, c_{viaje}), donde cada valor representa la cantidad de misioneros y caníbales que suben al bote:

- **Acción** $(1,0)$**:** Transportar exactamente un misionero.
- **Acción** $(2,0)$**:** Transportar dos misioneros.
- **Acción** $(0,1)$**:** Transportar exactamente un caníbal.
- **Acción** $(0,2)$**:** Transportar dos caníbales.

- **Acción** $(1,1)$**:** Transportar un misionero y un caníbal conjuntamente.

Cualquier combinación de pasajeros que supere los dos integrantes, que intente mover cero tripulantes o que intente trasladar más personas de las que físicamente se encuentran presentes en la orilla de origen queda completamente descartada del conjunto de acciones válidas, evitando así la generación de ramas inválidas en el árbol de búsqueda.

3.5. Función de Transición

La función de transición, denotada formalmente como $T(s, a)$, es el mecanismo matemático y lógico que define cómo evoluciona el sistema al aplicar una acción válida. Esta función toma como entradas un estado actual $s = (m_{izq}, c_{izq}, b)$ y una acción legal $a = (m_{viaje}, c_{viaje})$, y genera como resultado un nuevo estado sucesor $s' = (m'_{izq}, c'_{izq}, b')$.

El comportamiento de la función de transición depende directamente de la posición actual del bote (b), ya que determina si el viaje se realiza desde la orilla de origen hacia el destino o viceversa, de tal manera que, si **el bote se encuentra en la orilla izquierda (**$b = 1$) la embarcación parte de la izquierda hacia la derecha, los pasajeros seleccionados abandonan la orilla de origen. Por lo tanto, la función resta los valores de la acción a las poblaciones actuales y cambia la posición del bote a 0 generando el nuevo estado:

$$s' = (m_{izq} - m_{viaje}, c_{izq} - c_{viaje}, 0)$$

Si el bote se encuentra en la orilla derecha ($b = 0$) la embarcación realiza el viaje de retorno desde la derecha hacia la izquierda, los pasajeros que regresan se reincorporan a la orilla de origen. Por lo tanto, la función suma los valores de la acción a las poblaciones actuales y cambia la posición del bote a 1 generando el nuevo estado:

$$s' = (m_{izq} + m_{viaje}, c_{izq} + c_{viaje}, 1)$$

3.6. Restricciones

Para garantizar la validez del estado s' generado tras cada transición, el sistema implementa un conjunto de filtros lógicos que impiden la exploración de caminos inviables dentro del espacio de búsqueda. En primer lugar, se evalúan los límites físicos del entorno, asegurando que las cantidades tanto de misioneros como de caníbales se mantengan siempre dentro de un intervalo cerrado de cero a tres en la orilla de origen, mientras que la posición de la embarcación conserve su naturaleza estrictamente binaria para reflejar su ubicación actual.

El filtro más importante menciona que de manera simultánea en ambas orillas, la cantidad de misioneros nunca sea superada por la de caníbales siempre que haya al menos un misionero presente. Si una acción genera un estado que infringe este equilibrio en cualquiera de las dos orillas se lo descarta de forma inmediata, asegurando de este modo que el algoritmo avance exclusivamente a través de estados válidos y bajo a restricciones antes establecidas.

3.7. Criterios de Solución

Para que una secuencia de pasos sea considerada una solución válida y satisfactoria dentro del problema, se deben cumplir determinados criterios teóricos y de desempeño. En primer lugar, la solución debe constituir una trayectoria finita y ordenada de acciones legales, $P = (a_1, a_2, \dots, a_k)$, que aplique sucesivamente la función de transición sobre el estado inicial para alcanzar el objetivo, garantizando que todos los estados intermedios pertenezcan al espacio de estados válidos ($s_i \in S_{válidos}$):

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \dots \xrightarrow{a_k} s_k = s_{IsGoal}$$

Asimismo, la evaluación de la solución contempla la eficiencia en el uso de recursos informáticos durante el proceso de búsqueda. Para ello, se consideran métricas de desempeño clave como la cantidad total de nodos generados y expandidos en la memoria, el tiempo de ejecución requerido para encontrar la ruta y la profundidad máxima alcanzada en el árbol de exploración. De esta manera, el criterio de solución no solo exige garantizar un camino seguro y libre de ciclos, sino también proporcionar una base para analizar y comparar el comportamiento operativo de los algoritmos de búsqueda implementados.

4. Implementación en Python

4.1. Representación del Estado y Movimientos Permitidos del Bote

La primera parte de la implementación recae en la modelación de la información y la dinámica operativa. El sistema define el estado inicial mediante la tupla ESTADO_INICIAL = (3, 3, 1) y el estado meta como ESTADO_OBJETIVO = (0, 0, 0), donde cada configuración almacena de forma compacta las poblaciones en la orilla de origen y la posición binaria de la embarcación.

Para los desplazamientos, el código establece el conjunto cerrado MOVIMIENTOS_PERMITIDOS_BOTE, el cual contiene todas las combinaciones autorizadas de tripulantes que pueden cruzar el río, abarcando desde el traslado

individual de un misionero o caníbal hasta duplas combinadas.

4.2. Módulo de Validación de Estados

El módulo de validación de datos actúa como filtro de estados donde intercepta cualquier configuración candidata para certificar su viabilidad física y lógica antes de integrarla al árbol de búsqueda. Este componente se sustenta en dos funciones interconectadas dentro del sistema. Por un lado, la función es_orilla_segura evalúa de forma individual una orilla específica, estableciendo que si la cantidad de misioneros es igual a cero la orilla es segura, mientras que de haber presencia misionera exige que su número sea mayor o igual al de los caníbales para evitar escenarios de riesgo. Por otro lado, la función es_estado_valido procesa el estado global descomponiéndolo en las variables misioneros_izq, canibales_izq y posicion_bote, comprobando rigurosamente que los valores poblacionales se mantengan dentro del intervalo permitido de cero a tres y que la posición de la embarcación conserve su naturaleza binaria. Asimismo, este módulo deduce de manera automática la distribución complementaria para la orilla opuesta y aplica la condición de seguridad a ambas riberas de forma simultánea, descartando de inmediato cualquier configuración infractora para evitar la propagación de ramas inviables.

4.3. Módulo de Sucesores y Función de Transición

La función obtener_sucesores modela el desplazamiento físico evaluando el estado actual frente al conjunto de movimientos permitidos. Dependiendo de la ubicación del bote, el sistema comprueba primero la disponibilidad real de pasajeros en la orilla de partida; posteriormente, aplica las operaciones aritméticas correspondientes para actualizar las poblaciones y alternar la posición de la embarcación, retornando únicamente los nuevos estados que superan satisfactoriamente los filtros de validez.

4.4. BFS

La Búsqueda en Anchura (BFS) opera bajo un enfoque de exploración sistemática por niveles o capas concéntricas, examinando exhaustivamente todos los estados situados a una distancia de k pasos antes de avanzar hacia los niveles más profundos. En su implementación práctica, el motor se apoya en una estructura de datos de tipo FIFO (First-In, First-Out) utilizando una cola optimizada mediante cola = deque([inicial]), acompañada de un conjunto visitados = set([inicial]) y un diccionario padre = {inicial: None} para rastrear el camino. Durante cada iteración,

el algoritmo extrae el nodo ubicado al frente con cola.popleft(), verificando de inmediato si coincide con el ESTADO_OBJETIVO. Si no es así, el estado se expande mediante la función obtener_sucesores(estado), y cada sucesor inédito se marca como visitado, se encola al final de la estructura y registra su antecesor. Gracias a este avance uniforme por capas, BFS garantiza matemáticamente la optimalidad de la ruta, asegurando que el primer camino encontrado posea el costo mínimo indispensable de movimientos.

4.5. DFS

Por su parte, la Búsqueda en Profundidad (DFS) adopta una estrategia de exploración vertical y exhaustiva, priorizando adentrarse lo más hondo posible a lo largo de una única rama del árbol de decisiones hasta agotar sus alternativas o toparse con estados ya visitados, momento en el cual activa el mecanismo de retroceso o backtracking. En el código, este comportamiento se gestiona mediante una estructura de tipo LIFO (Last-In, First-Out) representada como una pila explícita con pila = [inicio], un conjunto de control visitados = {inicio} y un diccionario de rastreo padre = {inicio: None}. Operativamente, el algoritmo extrae el elemento del tope de la estructura usando pila.pop(), evalúa si representa la meta y, en caso contrario, obtiene sus ramificaciones con obtener_sucesores(estado) para apilar los nuevos estados descubiertos. En términos teóricos, DFS suele requerir menos memoria que BFS porque mantiene una frontera de exploración más reducida. Sin embargo, en esta implementación también se conservan un conjunto de estados visitados y un diccionario de padres, por lo que el espacio utilizado depende igualmente de la cantidad de estados descubiertos. Además, DFS no garantiza encontrar la ruta más corta.

4.6. Métricas y Rendimiento

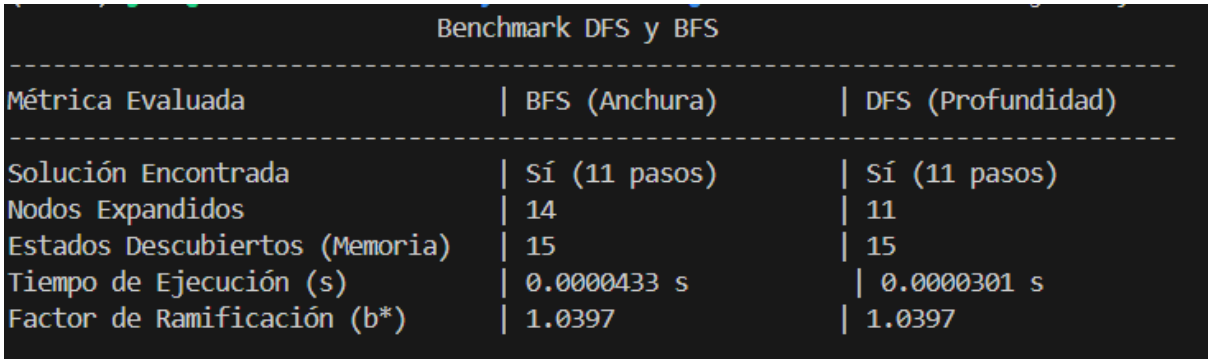
El módulo de recolección de métricas y rendimiento se encarga de instrumentar de forma cuantitativa el comportamiento operativo de los algoritmos de búsqueda, permitiendo contrastar con absoluta precisión la eficiencia entre BFS y DFS. Durante la ejecución, el sistema registra variables clave como el contador de nodos_expandidos, el tamaño total del conjunto de visitados para estimar el volumen de estados descubiertos en memoria y la medición del tiempo transcurrido mediante time.perf_counter(). Asimismo, a través de la función auxiliar calcular_factor_ramificacion_efectivo, el módulo procesa la profundidad final de la ruta y el total de nodos explorados para aproximar matemáticamente el factor de ramificación efectivo ($b*$) mediante un algoritmo de búsqueda binaria. Finalmente, la función ejecutar_benchmark consolida y organiza todos estos indicadores en una

tabla comparativa estructurada en consola, ofreciendo una base empírica sólida para evaluar las ventajas operativas y las limitaciones de cada estrategia de exploración.

5. Resultados Experimentales

Los resultados obtenidos en el benchmark permiten observar las diferencias entre los algoritmos de búsqueda en anchura (BFS) y búsqueda en profundidad (DFS), tanto en la forma en que recorren los estados como en los recursos que utilizan para encontrar una solución. Aunque ambos algoritmos consiguieron llegar al estado objetivo en 11 pasos, su comportamiento durante la exploración fue diferente. Esto se debe principalmente a la estrategia que utiliza cada uno para recorrer el espacio de búsqueda y seleccionar los siguientes estados que debe evaluar.

En cuanto al número de nodos expandidos, BFS registró 14, mientras que DFS solamente necesitó expandir 11. Esta diferencia se explica porque BFS explora los estados por niveles, es decir, primero revisa todas las posibilidades que se encuentran a una misma profundidad antes de avanzar hacia el siguiente nivel. Por esta razón, puede llegar a examinar estados que no forman parte del camino final, pero que son necesarios para garantizar que encuentra la solución más corta. Por otro lado, DFS continúa explorando una misma rama hasta llegar a un estado sin posibilidades de avance o encontrar el objetivo. En este caso, esa estrategia le permitió alcanzar la solución con menos nodos expandidos, ya que encontró un camino válido sin necesidad de recorrer tantas alternativas. Esto también se refleja en el tiempo de ejecución: BFS tardó 0.0000433 segundos, mientras que DFS necesitó 0.0000301 segundos. Aunque ambos tiempos son bastante pequeños, DFS presentó una ejecución ligeramente más rápida, principalmente por la menor cantidad de nodos que tuvo que expandir.



Benchmark DFS y BFS

Métrica Evaluada	BFS (Anchura)	DFS (Profundidad)
Solución Encontrada	Sí (11 pasos)	Sí (11 pasos)
Nodos Expandidos	14	11
Estados Descubiertos (Memoria)	15	15
Tiempo de Ejecución (s)	0.0000433 s	0.0000301 s
Factor de Ramificación (b*)	1.0397	1.0397

Figura 1. Resumen de métricas de rendimiento y eficiencia operativa entre BFS y DFS.

Respecto a los resultados asintóticos, las diferencias entre ambos algoritmos se relacionan con la manera en que aumenta el costo de búsqueda a medida que crece el

espacio de estados. BFS presenta una complejidad temporal y espacial de $\left(O\left(b^{d}\right)\right)$, donde b representa el factor de ramificación y d la profundidad de la solución más cercana. Por su parte, DFS presenta una complejidad temporal de $\left(O(b^{m})\right)$ y espacial de $\left(O(bm)\right)$ en el análisis tradicional de búsqueda en árboles, donde m representa la profundidad máxima del espacio de búsqueda. No obstante, estas complejidades espaciales corresponden al análisis teórico y pueden variar según la implementación. En este caso, ambos algoritmos utilizan un conjunto de estados visitados y un diccionario de padres, estructuras que también requieren memoria para conservar la información de los estados descubiertos.

Un aspecto importante es el consumo de memoria. Generalmente, BFS requiere más espacio porque debe almacenar los estados pendientes de explorar, especialmente cuando existen muchas posibilidades en un mismo nivel. DFS, en cambio, mantiene una frontera de exploración más reducida al profundizar por una misma rama. No obstante, en este benchmark ambos algoritmos registraron 15 estados descubiertos. Este resultado indica que la cantidad de estados almacenados en el conjunto de visitados fue la misma, pero no significa necesariamente que ambos utilizaran idéntica cantidad de memoria RAM. Para establecer esa diferencia sería necesario medir directamente el consumo máximo de memoria de cada implementación.

También se puede observar que ambos algoritmos presentan un factor de ramificación efectivo de 1.0397. Este valor indica que, considerando los 15 estados descubiertos y la profundidad de 11 pasos, el crecimiento efectivo del árbol de búsqueda fue reducido. Esto se relaciona con las características particulares del problema, en el que las restricciones sobre los movimientos limitan considerablemente las transiciones disponibles entre estados. Por esa razón, aunque los algoritmos utilizan estrategias distintas, terminaron explorando una cantidad relativamente similar de estados y encontraron la solución a la misma profundidad.

Comparativa de Caminos (BFS vs DFS)

Paso	Camino BFS	Camino DFS
0	(3, 3, 1)	(3, 3, 1)
1	(3, 1, 0)	(2, 2, 0)
2	(3, 2, 1)	(3, 2, 1)
3	(3, 0, 0)	(3, 0, 0)
4	(3, 1, 1)	(3, 1, 1)
5	(1, 1, 0)	(1, 1, 0)
6	(2, 2, 1)	(2, 2, 1)
7	(0, 2, 0)	(0, 2, 0)
8	(0, 3, 1)	(0, 3, 1)
9	(0, 1, 0)	(0, 1, 0)
10	(1, 1, 1)	(0, 2, 1)
11	(0, 0, 0)	(0, 0, 0)

Figura 2. Secuencia detallada de estados (tuplas de orilla y posición del bote) generados paso a paso por BFS y DFS.

Finalmente, es importante analizar si DFS garantizó encontrar el camino más corto. La respuesta es no, ya que DFS no garantiza la optimalidad de la solución: al explorar una rama hasta el fondo, puede encontrar un objetivo que esté a mayor profundidad, aunque exista otro camino más corto. En este caso particular, ambos algoritmos encontraron una solución de 11 pasos, pero esto no significa que DFS garantice ese resultado en cualquier situación. De hecho, al comparar los caminos obtenidos, se observa que existen diferencias en algunos estados intermedios, por ejemplo, en el paso 1 y en el paso 10, aunque ambos recorridos terminan llegando al mismo estado objetivo. BFS, por su parte, sí garantiza encontrar el camino más corto cuando todos los movimientos tienen el mismo costo, precisamente porque explora los estados de menor profundidad antes de avanzar hacia los más profundos.

6. Conclusiones

- La formulación matemática del problema de los Misioneros y Caníbales permitió representar de manera estructurada un entorno de búsqueda mediante estados discretos, acciones permitidas y restricciones de seguridad. La utilización de tuplas para identificar la cantidad de misioneros, caníbales y la posición del bote facilitó la construcción del espacio de estados, mientras que las funciones de validación y transición garantizaron que los movimientos generados respetaran las condiciones establecidas. De esta manera, se consiguió transformar un problema lógico en un modelo computacional que puede ser explorado sistemáticamente mediante algoritmos de búsqueda ciega.
- La implementación de BFS y DFS en Python permitió comprobar cómo dos estrategias diferentes pueden resolver un mismo problema utilizando mecanismos de exploración distintos. Ambos algoritmos encontraron una solución válida de 11 movimientos, aunque presentaron diferencias en algunos estados intermedios de sus recorridos. En este caso, BFS permitió confirmar que 11 movimientos corresponden a la solución más corta, mientras que DFS también alcanzó esa misma profundidad sin contar con una garantía general de optimalidad. Esto demuestra que encontrar una solución óptima mediante DFS puede ocurrir dependiendo del orden de exploración, pero no constituye una propiedad garantizada del algoritmo.
- Los resultados experimentales evidenciaron que DFS presentó un menor número de nodos expandidos, con 11 frente a los 14 registrados por BFS, además de un menor tiempo de ejecución en la prueba realizada. Este comportamiento se relaciona con su estrategia de profundizar por una rama y alcanzar directamente el objetivo sin explorar tantas alternativas. Por el

contrario, BFS necesitó examinar más estados debido a su recorrido por niveles, pero este comportamiento le permitió garantizar la obtención del camino más corto. Asimismo, ambos algoritmos registraron 15 estados descubiertos y un factor de ramificación efectivo de 1.0397, lo que refleja que las restricciones del problema redujeron las posibilidades de expansión y mantuvieron relativamente pequeño el espacio de búsqueda.

- Finalmente, se determinó que la eficiencia de BFS y DFS depende de las características del problema y de los objetivos que se persigan durante la búsqueda. BFS resulta especialmente apropiado cuando es indispensable garantizar una solución con el menor número de movimientos, mientras que DFS puede ser conveniente cuando se busca explorar una alternativa hasta encontrar una solución válida sin priorizar necesariamente la optimalidad. Aunque en este ejercicio DFS obtuvo mejores resultados en cantidad de nodos expandidos y tiempo de ejecución, el tamaño reducido del espacio de estados no permite generalizar esa ventaja a problemas de mayor dimensión. Por tanto, el desarrollo permitió comprender que la selección de una estrategia de búsqueda debe considerar conjuntamente la complejidad computacional, el consumo de recursos y las garantías que ofrece cada algoritmo.